

FORMATION PROFESSIONNELLE · 3 JOURS



DOCKER

Embarquement immédiat

Construire, sécuriser et déployer une application réelle — les mains dans le cambouis.

JOUR 1

Prendre la mer

JOUR 2

Gréer le navire

JOUR 3

La haute mer

Trois jours, une régates, une application réelle

- ▶ **Jour 1 — Prendre la mer.** Piloter, réparer et fabriquer des conteneurs ; écrire un vrai Dockerfile applicatif et l'optimiser (×8 plus léger).
- ▶ **Jour 2 — Gréer le navire.** Registry privé, volumes, réseaux front/back, puis toute la stack en une commande avec Compose.
- ▶ **Jour 3 — La haute mer.** Cluster Swarm 3 nœuds auto-réparant, mise à jour sans coupure, sécurité, CI/CD et le Grand Prix final.
- ▶ **Un fil rouge unique.** « ShipFast », un raccourcisseur d'URL, enrichi mission après mission jusqu'à la prod.
- ▶ **Une méthode.** 70 % clavier, 30 % oral. On casse, on répare. Tout converge vers le fil rouge.

« Jeudi 16h30, cette application tournera chez vous sur un cluster de 3 serveurs, et vous la mettrez à jour sans coupure. »

ShipFast — un raccourcisseur d'URL, de bout en bout

PROXY

nginx 1.30

- Point d'entrée unique
- Port mapping
- Reverse proxy

API

Node.js 24

- Crée les liens courts
- Dockerfile, multi-stage
- Healthcheck, env

DB

PostgreSQL 18

- Stocke les liens
- Volumes nommés
- Sauvegarde/restauration

CACHE

Redis 8

- Compte les clics
- DNS interne
- Isolation réseau

La même application vous suit 3 jours. Chaque notion l'enrichit : image → volume → réseau → compose → cluster.

La Docker League — la régates des capitaines

- ▶ **Chaque mission rapporte des nœuds.** 10 par TP réussi dans le temps, +5 au premier, +5 par défi bonus, 15 pour un challenge.
- ▶ **L'entraide paie.** +5 nœuds pour un « coup de main » à un participant bloqué — la compétition reste bon enfant.
- ▶ **Trois grades à conquérir.** Moussaillon au départ, Second (70 nœuds), Capitaine (160), Docker Captain (260) à la clôture.
- ▶ **Un tableau de score partagé.** Affiché à chaque débrief ; renommez votre profil Teams « Prénom — 0 nœud ».

Objectif : rythme, énergie et mémorisation. On récompense la maîtrise, jamais la vitesse seule.

MODULE 0

Un mot, quatre choses



Séparer les couches avant de parler licences

- Docker Desktop, Engine, containerd, runc : ce n'est pas la même chose
- Ce qui est gratuit, ce qui se paie en 2026
- Le standard OCI : on apprend un standard, pas un produit

Derrière le mot « Docker », quatre briques distinctes

PRODUIT	MOTEUR	RUNTIME	BAS NIVEAU
Docker Desktop • GUI poste de travail • VM + confort • Licence au-delà des seuils	Docker Engine (Moby) • daemon + CLI + API • Ce qu'on met en serveur • Apache 2.0 — gratuit	containerd (CNCF) • Cycle de vie conteneurs • Standard de Kubernetes • Open source	runc + OCI • namespaces + cgroups • Crée le conteneur • Image OCI = portable partout

Une image construite ici tourne sur Docker, Podman, Kubernetes — partout. C'est ça, le standard OCI.

Ce qui est gratuit, ce qui se paie (2026)

Libre et gratuit — partout

- **Docker Engine** serveurs Linux, prod incluse
- **containerd, runc, CLI**
- **Dockerfile & Compose (plugin)**
- **Rancher Desktop** Apache 2.0, sans seuil
- **Podman + Podman Desktop** rootless

Sous conditions / payant

- **Docker Desktop** gratuit si < 250 salariés ET < 10 M\$ de CA
- **Usage perso, éducation, OSS** gratuit
- **Entités gouvernementales** toujours payant
- **Sinon** abonnement par utilisateur/mois

La vraie question n'est pas « Docker est-il payant ? » — c'est « que met-on sur les postes ? »

Le bon outil au bon endroit

SERVEUR DE PROD

Docker Engine

- Libre, gratuit, éprouvé
- Sans orchestrateur

CLUSTER K8S

containerd

- Directement
- Le standard de fait

POSTE DEV

Desktop / Rancher / Podman

- Desktop si licence OK
- Rancher = gratuit
- Podman = sécurité

CI / CD

Builders sans démon

- Podman rootless
- Kaniko / buildx
- Vu au Jour 3

On n'apprend pas un produit cette semaine — on apprend un standard.

JOUR 1

Prendre la mer

Du premier conteneur à l'image optimisée

- Piloter, réparer et fabriquer des conteneurs
- Écrire son premier Dockerfile applicatif
- Optimiser une image : multi-stage, alpine, non-root (×8 plus léger)



Machine virtuelle vs conteneur

Machine virtuelle

- **Virtualise le matériel** hyperviseur + OS invité complet
- **Lourd** des Go de RAM par VM
- **Lent** démarrage en minutes
- **Isolation forte** mais coûteuse

Conteneur

- **Virtualise l'OS** namespaces + cgroups du noyau
- **Léger** des Mo, noyau partagé
- **Rapide** démarre en millisecondes
- **Un conteneur = un processus** isolé

Le conteneur partage le noyau de l'hôte. C'est toute la différence — et tout le gain.

Que se passe-t-il quand vous tapez docker run ?

- ▶ **docker (client)**. Envoie votre commande à l'API REST du démon.
- ▶ **dockerd (démon)**. Reçoit l'ordre, tire l'image, orchestre.
- ▶ **containerd**. Gère le cycle de vie du conteneur.
- ▶ **runc**. Crée réellement le conteneur : namespaces + cgroups.

LA CHAÎNE, EN DIRECT

```
# À montrer en live
docker system info
ps aux | grep -E 'dockerd|containerd'

# le conteneur est un vrai
# processus de l'hôte
docker top web
```

Vrai marin ou terrien ? — échauffement

À l'oral, réponses simultanées dans le chat au « 3, 2, 1, go ». 2 nœuds par bonne réponse.

1 Un conteneur embarque-t-il un noyau Linux ?

2 Que se passe-t-il quand le PID 1 d'un conteneur se termine ?

3 `docker run ubuntu` puis `docker ps` : combien de conteneurs ?

4 Une image Docker, c'est modifiable ?

5 Docker Desktop est-il gratuit pour votre entreprise ?

6 Citez deux orchestrateurs de conteneurs.

Les 10 commandements du conteneur

FORMAT

TP guidé individuel, débrief collectif.

ON CONSTRUIT

Un serveur web réel, piloté de bout en bout — comprendre au clavier qu'un conteneur = un processus.

5 QUESTIONS

Notez vos réponses, on corrige ensemble à 11h05.

AU CLAVIER

```
# La toute première fois
docker run hello-world

docker run -it ubuntu bash

docker ps -a          # Exited

# Un vrai serveur en arrière-plan
docker run -d --name web -p 8080:80 \
    nginx:1.30-alpine
docker exec -it web sh
docker logs -f web    |    docker stats
```

Le Détective — 5 conteneurs en détresse

FORMAT

Énigmes individuelles, entraide encouragée.

ON CONSTRUIT

Le réflexe de diagnostic `ps -a` → `logs` → `inspect` : ce qui distingue un utilisateur d'un pro.

PIÈGE

137 = OOM · 139 = segfault · 143 = SIGTERM.

AU CLAVIER

```
# Vous etes d'astreinte. 5 tickets.
docker run -d --name t1 ubuntu
docker run -d --name t5 postgres:18-alpine
docker ps    # t5 a disparu !

# Interdit de demander la solution
# avant un logs ou un inspect
docker logs t5
docker inspect -f '{{.State.ExitCode}}' t5
```

L'image à la main : commit, diff... et pourquoi c'est mal

FORMAT

TP guidé, puis « le procès de docker commit ».

ON CONSTRUIT

Votre première image personnalisée, capturée depuis un conteneur — pour comprendre pourquoi le Dockerfile existe.

VERDICT

Il nous faut une recette texte, rejouable : le Dockerfile.

AU CLAVIER

```
docker exec -it atelier sh
  echo '<h1>...' > ../index.html
  apk add --no-cache figlet
docker diff atelier    # A/C/D

docker commit atelier monsite:fait-main
docker history monsite:fait-main
# couche muette : non reproductible,
# non versionnable, non auditable
```

Docker Hub : tags, variantes et limites de pull

- ▶ **Anatomie d'une image officielle.** Tags, variantes alpine/slim, docs des variables d'env.
- ▶ **Official Image vs Verified Publisher.** Savoir à qui l'on fait confiance.
- ▶ **Épingler un tag précis.** postgres:18-alpine — jamais latest en prod.
- ▶ **Les limites de pull anonymes.** 100 pulls/6 h par IP → d'où le registry privé de demain.

BONNES HABITUDES

```
docker pull node:24-alpine
docker pull postgres:18-alpine

# s'authentifier relève la limite
docker login

# épingler par digest = reproductible
node:24-alpine@sha256:...
```


Une image, c'est un empilement de couches

- ▶ **Chaque instruction = une couche.** En lecture seule, empilée sur la précédente.
- ▶ **Les couches sont partagées.** Tirer node:24 une fois sert à toutes vos images.
- ▶ **Le cache suit l'ordre.** Dépendances d'abord, code ensuite → rebuilds quasi instantanés.
- ▶ **Le conteneur ajoute une couche d'écriture.** Jetable, perdue à sa destruction → d'où les volumes.

LES COUCHES, DU HAUT VERS LA BASE

```
CMD ['node', 'server.js']  
COPY server.js .  
RUN npm ci  
COPY package.json .  
FROM node:24-alpine  
  
# + couche d'écriture (conteneur)
```

Premier Dockerfile : la carte de visite

FORMAT

TP individuel court et gratifiant.

ON CONSTRUIT

Votre page web perso empaquetée en image versionnée — installer la boucle write → build → run → itère.

LE MOMENT « C'EST EN LIGNE ! »

Sur la VM publique, la page est réellement accessible.

AU CLAVIER

```
# moncv/Dockerfile — 2 lignes
FROM nginx:1.30-alpine
COPY index.html /usr/share/nginx/html/

docker build -t moncv:v2 .
docker run -d --name cv -p 8081:80 moncv:v2
# v1 ET v2 coexistent : versioning gratuit
```

Conteneuriser ShipFast : le Dockerfile applicatif

FORMAT

TP fil rouge individuel. IA interdite aujourd'hui.

ON CONSTRUIT

L'image de l'API du fil rouge : dépendances, code, port, commande de démarrage.

PIÈGE

COPY . . sans .dockerignore → node_modules local qui écrase tout.

AU CLAVIER

```
FROM node:24
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]

docker build -t shipfast:v1 .    # ~1,1 Go
```

Opération régime — l'image la plus légère gagne

FORMAT

Challenge chronométré, indices payants.

ON CONSTRUIT

La version prod de l'image : multi-stage, alpine, .dockerignore, non-root, cache maîtrisé.

VERDICT

docker images partagé : la plus légère qui passe les 3 tests gagne.

AU CLAVIER

```
# syntax=docker/dockerfile:1
FROM node:24-alpine AS build
COPY package*.json ./ ; RUN npm ci
COPY . . ; RUN npm run build

FROM node:24-alpine
USER node
CMD ["node", "dist/server.js"]
# de ~1,1 Go a ~140 Mo, non-root
```

L'image parfaite — la checklist à épingler

✓ Léger : multi-stage + base alpine/distroless

✓ En cache : dépendances avant le code

✓ Non-root : USER + COPY --chown

✓ Épinglé : tag par digest, jamais latest

✓ Sondé : HEALTHCHECK défini

✓ Propre : tini (PID 1) + forme exec

✓ Outillé : dive (radiographie des couches)

✓ Linté : hadolint sur le Dockerfile

De ~1,1 Go en root à ~140 Mo non-root, scannée et signée.

Clôture Jour 1 — 10 questions

1 Que devient un conteneur quand son PID 1 meurt ?

3 Premier réflexe quand un conteneur ne démarre pas ?

5 À quoi sert `.dockerignore` ?

7 Différence `CMD` vs `ENTRYPOINT` ?

2 Commande pour voir les conteneurs arrêtés ?

4 Pourquoi `docker commit` est-il déconseillé ?

6 Pourquoi copier `package.json` avant le code ?

8 Que gagne-t-on avec un build multi-stage ?

Classement de fin de journée, remise des grades, teaser du Jour 2.

JOUR 2

Gréer le navire



Registry, volumes, réseaux et Compose

- Un registry privé et la persistance par les volumes
- Isoler les services par le réseau : front / back
- Toute la stack en une commande avec Compose

Les données survivent au conteneur : les volumes

VOLUME NOMMÉ

Données de prod

- Géré par Docker
- Sauvegardable
- Base de données, registry

BIND MOUNT

Roi du développement

- Miroir d'un dossier hôte
- Rechargement à chaud
- Code source, config

TMPFS

En RAM pure

- Jamais écrit sur disque
- Éphémère
- Secrets, fichiers temporaires

Règle mémorisable : les volumes pour les données du conteneur, les bind mounts pour les fichiers de l'humain.

Registry privé & volumes : la persistance

FORMAT

TP guidés, crash test à l'appui.

ON CONSTRUIT

Un registry interne (contourne les limites de pull) et des données PostgreSQL qui survivent au conteneur.

BONUS CRASH TEST

Sauvegarde/restauration par dump.sql.

AU CLAVIER

```
docker run -d -p 5000:5000 --name reg registry:3
```

```
# la douleur, puis la solution
```

```
docker volume create pgdata
```

```
docker run -d --name db -v pgdata:/var/lib/\n    postgresql/data postgres:18-alpine
```

```
docker rm -f db    # on détruit...
```

```
# ...les données sont toujours là
```

Isoler par le réseau : l'architecture front / back

- ▶ **Réseau front.** Internet → proxy (nginx) → api. Un seul port publié (:8080).
- ▶ **Réseau back (--internal).** api ↔ db ↔ cache. Aucun accès Internet.
- ▶ **L'API a un pied dans chaque réseau.** Elle relie le public et les données.
- ▶ **La base et le cache ne sont JAMAIS joignables** depuis le proxy ni depuis Internet. DNS interne par nom de service.

LE CÂBLAGE À LA MAIN

```
docker network create front
docker network create --internal back

# db et cache : reseau back seulement
docker run -d --network back --name db ...

# api : les deux reseaux
docker run -d --network back --name api ...
docker network connect front api
```

Docker Compose : toute la stack en une commande

À LA MAIN

Fastidieux et fragile

- network create ×2
- run -d ×4 dans le bon ordre
- connect, -p, -e, -v...
- Rien n'est versionné

AVEC COMPOSE

Un fichier, une commande

- docker compose up -d
- réseaux + services créés
- ordre (depends_on)
- healthchecks vérifiés

Un seul fichier compose.yaml, versionné dans Git : la stack devient reproductible.

Compose : assembler ShipFast + « Casse-prod »

FORMAT

TP fil rouge, puis challenge de débogage.

ON CONSTRUIT

Toute la stack ShipFast dans un compose.yaml, puis réparer 5 sabotages en direct.

CASSE-PROD

5 sabotages listés dans le corrigé : à retrouver en équipe.

AU CLAVIER

```
services:
  proxy: { image: nginx:1.30-alpine,
    ports: ['8080:80'] }
  api:   { build: ., depends_on: [db,cache] }
  db:    { image: postgres:18-alpine,
    volumes: [pgdata:/var/lib/postgresql/data] }
  cache: { image: redis:8-alpine }
networks: [front, back] # back --internal
```

Faire grandir la stack

FORMAT

TP guidé, plusieurs ateliers courts.

ON CONSTRUIT

Profiles (Adminer), compose watch, variables .env, scaling — les gestes du quotidien.

BONUS

Portainer : le cockpit graphique de la stack.

AU CLAVIER

```
docker compose --profile debug up -d # Adminer
docker compose watch # hot reload
docker compose up -d --scale api=3

# variables d'environnement
env_file: .env
# jamais de secret en clair dans le YAML
```

Clôture Jour 2

1 Quelle différence entre un volume nommé et un bind mount ?

3 Comment un service en joint-il un autre par son nom ?

5 Pourquoi un registry privé en entreprise ?

7 Commande pour scaler un service à 3 répliques ?

2 À quoi sert un réseau --internal ?

4 Que fait depends_on dans Compose ?

6 Où ne JAMAIS mettre un secret ?

8 Qu'apporte un healthcheck ?

Grades mis à jour. Demain : la haute mer, l'orchestration et la production.

JOUR 3

La haute mer



Swarm, Kubernetes, CI/CD et production

- Un cluster de 3 nœuds qui s'auto-répare
- Mise à jour sans coupure — le clou des 3 jours
- Sécurité, CI/CD et le Grand Prix final

On ne dit plus quoi FAIRE, mais quoi MAINTENIR

Impératif (avant)

- « Démarre 3 conteneurs »
- Si l'un tombe à 3 h du matin... vous vous levez
- Vous gérez chaque étape
- Fragile à l'échelle

Déclaratif (orchestrateur)

- « Je veux TOUJOURS 3 répliques »
- Si l'un tombe le manager en recrée un
- Vous décrivez l'état voulu
- Vous dormez

C'est le modèle commun à Swarm ET Kubernetes.

Un cluster Swarm de 3 nœuds

FORMAT

TP sur Play with Docker (multi-nœuds).

ON CONSTRUIT

Un vrai cluster : un manager, deux workers, un service répliqué qui s'auto-répare.

AUTO-RÉPARATION

Tuez un conteneur : le manager en recrée un aussitôt.

AU CLAVIER

```
docker swarm init --advertise-addr <ip>
docker swarm join --token <...> <manager>:2377
docker node ls

docker service create --name web \
    --replicas 3 -p 8080:80 nginx:1.30-alpine
docker service ps web # repartitionné sur 3 nœuds
```

ShipFast en prod : secrets & rolling update

FORMAT

TP fil rouge sur le cluster.

ON CONSTRUIT

La stack déployée avec secrets Swarm et mise à jour progressive — sans coupure.

BONUS CHAOS

Coupez un nœud pendant la bascule : ça tient.

AU CLAVIER

```
echo 'motdepasse' | docker secret create pg_pwd -  
docker stack deploy -c stack.yaml shipfast
```

```
docker service update --image shipfast:v4 \  
  --update-order start-first \  
  --update-parallelism 1 shipfast_api
```

Mettre à jour sans une seule requête perdue

- ▶ **1 · v3 · v3 · v3.** 3 répliques en service.
- ▶ **2 · v4 · v3 · v3.** On démarre 1 v4 AVANT d'éteindre une v3 (start-first).
- ▶ **3 · v4 · v4 · v3.** Le healthcheck valide chaque nouvelle avant de continuer.
- ▶ **4 · v4 · v4 · v4.** Bascule terminée, zéro coupure.

LA PREUVE, EN DIRECT

```
# la sonde pendant la bascule
while true; do \
    curl -s -o /dev/null -w '%{http_code} ' \
    localhost:8080; done

200 200 200 200 200 200 ...
# que des 200. La page ne bronche pas.
```

Compose, Swarm, Kubernetes ou serverless ?

Simple et local

- **Compose** 1 machine, dev & petites prods
- **Swarm** cluster simple, courbe douce
- **Idéal pour apprendre** et beaucoup de vraies prods

Grande échelle

- **Kubernetes** le standard, riche et complexe
- **Serverless / managé** zéro serveur à gérer
- **Le bon choix** = le plus simple qui tient vos contraintes

Kubernetes a gagné la guerre de l'orchestration — mais on n'en a pas toujours besoin.

Kubernetes en 5 minutes : le vocabulaire

- ▶ **Pod.** La plus petite unité déployable : un ou plusieurs conteneurs.
- ▶ **Deployment.** « Je veux TOUJOURS N répliques » — le déclaratif, comme Swarm.
- ▶ **Service.** Un point d'entrée stable et un DNS interne pour vos pods.
- ▶ **On reste OCI.** kompose convertit un compose, k3d monte un cluster local, k9s pilote.

LE PONT VERS KUBERNETES

```
# convertir un compose en manifestes K8s
kompose convert -f compose.yaml

# cluster local en 30 s
k3d cluster create formation
kubectl get pods
k9s    # le cockpit terminal
```

Deux moments, deux familles d'outils

AVANT L'EXÉCUTION

« L'image a-t-elle une faille connue ? »

- Trivy / Grype — scan CVE
- Dockle — bonnes pratiques
- hadolint — Dockerfile
- Docker Scout — priorise

PENDANT L'EXÉCUTION

« Un comportement anormal ? »

- Falco — détection runtime
- Appels système (eBPF)
- Alertes temps réel
- Complète le scan, pas l'inverse

Une image au scan parfait peut être compromise à l'exécution. Il faut les deux.

Durcissement production

FORMAT

3 ateliers courts + démos.

ON CONSTRUIT

Limites de ressources, rotation des logs, scan
Trivy : rendre la stack digne de la prod.

DÉMOS

Dozzle & cAdvisor (cockpits), Falco (runtime).

AU CLAVIER

```
# limites : eviter l'OOM et le voisin bruyant
deploy: { resources: { limits: {
    cpus: '0.5', memory: 256M } } }

docker run --rm aquasec/trivy image shipfast:v2
docker scout recommendations shipfast:v2
logging: { options: { max-size: '10m' } }
```

La CI/CD avec GitHub Actions

FORMAT

TP guidé, workflow complet.

ON CONSTRUIT

Un pipeline build → scan → push : l'image se construit, se scanne et se publie toute seule.

RENDEZ-VOUS

hadolint, vu en Mission 6, entre en CI.

AU CLAVIER

```
jobs:
  build:
    - uses: hadolint/hadolint-action
    - uses: docker/build-push-action
    - uses: aquasec/trivy-action # bloquant
    - run: docker push ghcr.io/...
# le scan echoue -> le build echoue
```


Le Grand Prix — capstone en binômes

FORMAT

Challenge final chronométré, app inconnue.

ON CONSTRUIT

Une application jamais vue (« météo-du-port »)
mise en prod de zéro : la preuve que la
méthode se transfère.

BARÈME

Image, sécurité, déploiement, propreté —
points par critère.

AU CLAVIER

```
cd meteo-du-port    # Python/Flask + Redis
# a vous de jouer :
# - Dockerfile multi-stage non-root
# - compose / stack + healthcheck
# - scan Trivy propre
# - déploiement qui repond en :8090
```

MODULE IA

Docker + IA : le copilote du conteneur



Les outils 2026 qui accélèrent — sans lâcher la barre

- Générer, déboguer et sécuriser plus vite
- Gordon, docker init, Copilot, Docker Scout
- La règle d'or : l'IA propose, vous validez

Gordon — l'agent IA de Docker (docker ai)

- ▶ **Intégré nativement.** CLI docker ai et Docker Desktop ; accès au CLI, au shell et aux fichiers du projet.
- ▶ **Human-in-the-loop.** Il affiche l'action qu'il veut exécuter ; vous approuvez ou refusez.
- ▶ **GA en 2026.** Docker Desktop 4.74 ; inclus gratuitement au tier de base (quotas de questions).
- ▶ **Ce qu'il sait faire.** Expliquer une erreur, optimiser un Dockerfile, conteneuriser une app, piloter les conteneurs.

EN DÉMO

```
docker ai
```

```
# des prompts qui font mouche :
```

```
"Why won't my container start?"
```

```
"Optimize this Dockerfile"
```

```
"Containerize this Node.js project"
```

```
"Stop the container using most RAM"
```

docker init & Copilot : générer les fichiers Docker

docker init

Officiel, GA

- Dockerfile + compose
- + .dockerignore + README
- Détecte le langage
- Multi-stage déjà propre

@docker (Copilot)

Dans Copilot Chat

- Génère les assets Docker
- Explique les concepts
- Peut ouvrir une PR
- Node, Python, Java

Le réflexe

Générer ≠ finir

- Relire les couches
- Épingler le tag
- Passer non-root
- Puis scanner

Comparez le Dockerfile de docker init avec votre shipfast.v2 fait main : vous savez maintenant le relire.

Docker Scout : la sécurité augmentée

- ▶ **Analyse continue.** CVE + SBOM de vos images, en local ou en CI.
- ▶ **L'IA priorise.** Non pas « 200 failles » mais « change ta base et tu en effaces 180 ».
- ▶ **Score EPSS.** Cible ce qui est réellement exploitable, pas le bruit.
- ▶ **Le pont avec la Mission 15.** Là où Trivy liste, Scout propose l'action corrective.

PRIORISATION INTELLIGENTE

```
docker scout cves shipfast:v2
docker scout recommendations shipfast:v2

# ne garder que l'exploitable
docker scout cves --epss-score 0.5 \
  --only-severity critical,high shipfast:v2
```

L'IA dans Docker : Model Runner & MCP Toolkit

DOCKER MODEL RUNNER

Un LLM en local, comme une image

- docker model run ai/smollm2
- OCI Artifacts, API OpenAI
- GA depuis Desktop 4.42
- Gratuit, open source

MCP CATALOG / TOOLKIT

Des outils d'agents, conteneurisés

- 100+ serveurs MCP signés
- Grafana, Stripe, Neo4j...
- Isolés dans des conteneurs
- Pour Gordon, Claude, Copilot

L'IDÉE

Docker sécurise l'IA

- Chaque outil isolé
- Signé et vérifié
- Fonctionne hors ligne
- À mentionner en ouverture

Docker traite modèles et outils d'IA comme des citoyens de première classe — au même titre que les conteneurs.

Les 4 règles de sécurité IA à graver

✓ Jamais de secret dans un prompt — il transite par des tiers

✓ L'IA hallucine tags & flags — vérifiez chaque référence (slopsquatting)

✓ On relit ET on scanne TOUJOURS le Dockerfile généré

✓ Méfiez-vous des données « anodines » — un LABEL piégé (faille DockerDash 2026)

✓ Pas de root, pas d'ENV secret, pas d'ADD d'URL distante

✓ docker scout cves systématique après génération

L'IA est un copilote, pas le pilote. Elle propose ; vous validez, testez, scannez.

« Le copilote à l'épreuve »

FORMAT

TP guidé en autonomie (optionnel / rapides).

ON CONSTRUIT

Le réflexe générer → relire → corriger → scanner sur une app inconnue.

LE MESSAGE

Ce sont VOS réflexes des 3 jours qui transforment sa proposition en image de prod.

AU CLAVIER

```
cd meteo-du-port
docker init          # ou : docker ai 'containerize this'

# RELIRE : root ? multi-stage ? tag epingle ?
docker build -t meteo:ia .
docker scout cves meteo:ia
docker scout recommendations meteo:ia
# quelle reco appliquer en premier (EPSS) ?
```


Votre environnement : une VM Linux, dans le navigateur

IDENTIQUE POUR TOUS

VM Linux OVHcloud

- Docker préinstallé
- Même comportement partout

ZÉRO INSTALLATION

SSH ou navigateur

- Terminal ttyd intégré
- Rien à configurer

PERSISTANT 3 JOURS

Votre ShipFast vous attend

- Du mardi au jeudi
- Snapshots quotidiens

TOUT EST PRÊT

Images pré-tirées

- Dépôt Git cloné
- Aucun proxy d'entreprise

Plan B universel : Play with Docker en 2 minutes si un poste est bloqué.

La cérémonie : classement, grades et cap sur la suite

- ▶ **Le podium de la Docker League.** Remise des grades : Moussaillon, Second, Capitaine, Docker Captain.
- ▶ **Les attentes cochées.** On reprend le tour de table du Jour 1 : chaque objectif validé publiquement.
- ▶ **Le plan d'action « lundi matin ».** Chacun repart avec 3 gestes à appliquer sur un projet réel.
- ▶ **Les ressources.** Cheatsheet des 3 jours, 50 astuces de pro, dépôt Git, checklist prod.

À EMPORTER

```
# votre kit de survie
- Playbook + guides (PDF)
- Depot GitHub de la formation
- Cheatsheet & 50 astuces
- Checklist production

# et le module IA 2026
- Gordon, Scout, docker init
```

À EMPORTER

En 3 jours, vous saurez



- Conteneuriser proprement n'importe quelle application
- Assembler une stack complète avec Compose
- Déployer en production et mettre à jour sans coupure
- Appliquer les bons réflexes de sécurité dès le départ
- Faire de l'IA un copilote — et relire ce qu'elle écrit

Bonne régate ! — Sparks Formation · Juillet 2026